
Named Entity Recognition With Transformer-Based Models and CRF

Entang Wang
7069521
enwa00001@stud.uni-saarland.de

Abstract

Named Entity Recognition (NER) is a fundamental task in natural language processing (NLP) that involves identifying and classifying entities such as names of people, organizations, locations, and other predefined categories within text. This project explores modeling hidden linguistic structures in short sentences and leveraging these structures for predictive tasks. We demonstrate the application of transformer-based neural network models for NER and investigate the enhancement of these models using statistical learning methods like Conditional Random Fields (CRF).

1 Introduction

Named Entity Recognition (NER) plays a pivotal role in semantic analysis, serving as a cornerstone for various NLP applications, including information retrieval, question answering systems, and knowledge extraction. By identifying and categorizing entities within text, NER facilitates a deeper understanding of language, enabling machines to process and interpret information more effectively.

Traditionally, NER tasks have been approached using rule-based systems [1, 2] and statistical models [3, 4, 5]. Rule-based systems rely on hand-crafted linguistic rules and lexicons to identify entities, while statistical models, such as Hidden Markov Models (HMMs) and Maximum Entropy Markov Models (MEMMs), utilize probabilistic methods to predict entity labels based on observed data. These approaches, however, often struggle with capturing complex contextual dependencies and require extensive feature engineering.

BERT for NER The advent of deep learning has revolutionized NER, with models like Bidirectional Encoder Representations from Transformers achieving state-of-the-art performance[6]. BERT[7] is a transformer-based model designed to understand the context of words in a sentence by considering both the left and right sides of a given word. Fine-tuning BERT for NER involves adapting the pre-trained BERT model to the specifics of an NER task, allowing it to leverage its pre-trained contextual understanding for the specialized task of identifying named entities in a given domain. This process has been shown to significantly improve accuracy and overall performance for NER tasks [8].

CRF for NER Conditional Random Field (CRF) is a class of statistical modeling methods often applied in pattern recognition and machine learning for structured prediction. In the context of NER, CRF is well-suited because it takes context into account, modeling the prediction as a graphical model that represents the presence of dependencies between the predictions. This allows CRF to effectively handle the sequential nature of language, capturing the relationships between neighboring words to improve entity recognition [3, 4, 5, 9].

2 Methodology

In this section, we outline the methodologies employed in the project, including data processing, model architectures, training procedures, and prediction strategies.

2.1 Data Processing

Our approach involves tokenizing sentences and aligning labels to ensure that each token corresponds accurately to its respective entity tag. We utilize the `Hugging Face datasets` library to load the dataset and the `transformers` library for tokenization.

The concrete steps are as follows:

1. Loading Dataset
We load the dataset using the `load_dataset` function, specifying the dataset name and split (e.g., 'train', 'validation', 'test').
2. Tokenization and Label Alignment
Using a tokenizer (here we use pretrained `roberta-base` tokenizer), we tokenize the input sentences. During this process, we align the NER tags with the tokenized words, ensuring that each token receives the correct label since the tokenizer may sometimes tokenize a word into several subtokens. In this circumstance, only the first subtokens will be allocated an NER tags and the rest subtokens will be ignored and assigned a label of -100. Special tokens like `<S>`, `<pad>`, and `</S>` introduced by the tokenizer are also assigned a label of -100 to be ignored during loss computation and NER prediction evaluation.
3. Saving the Processed Dataset
The tokenized and label-aligned dataset splits are saved to disk for efficient loading during the subsequent training and evaluation phases.

2.2 Model

We mainly explore four distinct model architectures in this project:

- **BERT_BASE**
This baseline model utilizes the pre-trained `roberta-base` model with a linear NER tag classifier. The pre-trained `roberta-base` model accepts a sequence of token input IDs along with corresponding attention masks indicating which token should be ignored and outputs a 768-dimensional logits sequence representing the probability distribution over NER tags for each token. Then, this logit sequence is passed to the linear classifier, which projects it into the space of NER tags. Predictions are independently generated for each token without explicitly modeling dependencies between token labels.
- **BERT_CRF**
This model extends **BERT_BASE** by incorporating a Conditional Random Field (CRF) layer on top of the RoBERTa encoder. Similar to **BERT_BASE**, this model first encodes the input sequence using the RoBERTa encoder. The contextual embeddings are then passed through a linear classifier to produce emission scores for each possible NER tag at each position. These emission scores are subsequently fed into a CRF layer. The CRF layer explicitly models the sequential dependencies between labels, capturing relationships such as which tags typically follow one another. During training, the CRF layer maximizes the likelihood of the correct sequence of tags, while during inference, it uses the Viterbi algorithm to decode the globally optimal sequence of labels, thus ensuring structural consistency in predictions.
- **MyBERT_BASE**
MyBERT_BASE is a custom-designed encoder-only transformer model inspired by BERT. Its mechanism includes:
 1. A word embedding layer converting token input IDs to a word embedding vector representation.
 2. A positional embeddings layer to inject token order information.

3. Multiple transformer encoder layers, each applying multi-head self-attention followed by position-wise feed-forward networks. Layer normalization and residual connections ensure stable learning and facilitate gradient flow.
4. A final linear classifier layer converts the transformer-generated embeddings into logits to predict the likelihood of each NER tag independently for each token.

This custom implementation serves as a baseline to evaluate the effectiveness of an implemented-from-scratch transformer architecture, which is used for the NER tagging task.

- MyBERT_CRF

Building upon MyBERT_BASE, this model adds a CRF layer on top of the custom transformer encoder. Similar to BERT_CRF, after obtaining embeddings and emission scores from the transformer, the CRF layer explicitly captures label dependencies and models the sequential structure of the labels. By integrating CRF, the model performs structured prediction, thus improving the coherence and reliability of the predicted sequences. The training and decoding processes (Viterbi algorithm) are identical to those described in BERT_CRF.

Item	Value
vocabulary size	50265
word embedding dimension	768
layers number	12
attention heads number	12
max context length	256

Table 1: Some parameters settings of MyBert

2.3 Training

In this section, we briefly describe the training process of this project.

To train the BERT_BASE, we use a relatively high learning rate to speed up its convergence. The parameters of the pretrained roberta-base model are frozen, and we only train the linear classifier parameters. We use the cross-entropy loss to optimize these parameters.

Similar to BERT_BASE, the parameters of roberta-base are also frozen in BERT_CRF. We use a smaller learning rate to prevent the unstable output at the training beginning period of the linear classifier from causing the CRF layer’s modeling of transfer probabilities to collapse quickly. We train the classifier and CRF layer at the same time, optimizing them with the negative log-likelihood loss calculated by the CRF layer.

The training methods of MyBERT_BASE and MyBERT_CRF are roughly the same. It is worth noting that since MyBERT_BASE has not been trained on a large scale corpus and captured enough semantic information, if we train it directly on a small corpus like CoNLL-2003, its performance will be unstable and fluctuate up and down 1. Therefore, it is foreseeable that its performance is clearly inferior to that of the pre-trained BERT_BASE model.

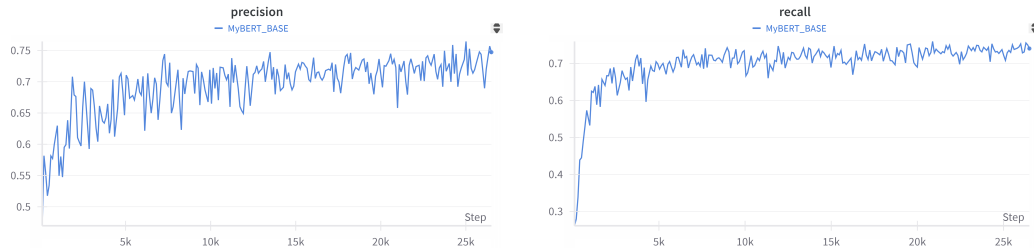


Figure 1: Left: Precision curve of MyBERT_BASE. Right: Recall curve of MyBERT_BASE. From the figure, we can see that, during the training process, the prediction performance of MyBERT_BASE is always fluctuating, especially the prediction precision.

2.4 Prediction

In this section, we focus on the main differences between the prediction methods for models that do not integrate a CRF layer and those that do.

For the standard transformer-based models (BERT_BASE and MyBERT_BASE), the prediction is performed independently for each token by selecting the NER tag with the highest probability from the output logits. Specifically, these models apply an argmax operation directly to the prediction scores computed by the final linear classifier, without explicitly modeling dependencies between adjacent token predictions.

In contrast, models equipped with a CRF layer (BERT_CRF and MyBERT_CRF) utilize structured prediction techniques. Instead of independently assigning tags to tokens, these models employ the Viterbi decoding algorithm to determine the optimal global sequence of tags.

3 Experiment

3.1 Dataset

In this project, we utilize the CoNLL-2003 dataset, which is a widely recognized benchmark for Named Entity Recognition (NER) tasks. The CoNLL-2003 dataset includes four main entity types: PER (person names), ORG (organizations), LOC (locations), and MISC (miscellaneous entities such as events, nationalities, or products). The NER tags include: O, B-PER, I-PER, B-ORG, I-ORG, B-LOC, I-LOC, B-MISC, I-MISC, where B stands for the beginning and I stands for inside. Each sentence in the dataset is splitted into words, and every word is assigned a corresponding tag. The dataset is split into training, validation, and test dataset. In our experiment, we use the training dataset for model training, the validation dataset for evaluating during the training process, and the test dataset for the post-trained evaluation.

3.2 Parameter Settings

The specific hyperparameters utilized in training our models are presented in Table 2. We fine-tuned these parameters based on extensive preliminary experiments, focusing primarily on the learning rate, training epochs, weight decay, and batch size.

The variation in the learning rate among the models is primarily due to differences in architecture complexity and whether the transformer is pretrained. The BERT_BASE model uses a relatively high learning rate ($1e-3$) since it freezes the pre-trained RoBERTa parameters, leaving only the linear classifier layer trainable. Therefore, we need a larger learning rate to allow the gradient to quickly pass through the deep network structure of BERT to have a more significant effect on this single classifier layer, thus facilitating rapid convergence with fewer trainable parameters. Conversely, the models employing the CRF layer, BERT_CRF and MyBERT_CRF, use smaller learning rates ($2.5e-4$ and $3e-5$, respectively) because structured prediction models typically require more cautious updates to converge stably; otherwise, the training will crush at an early period because of gradient explosion 2.

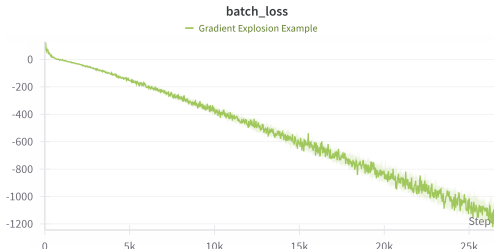


Figure 2: A training of MyBERT_CRF where gradient explosion occurs. The batch loss rapidly drops and shows no sign of convergence.

Regarding training epochs, we reduced the number of epochs for the BERT_CRF model from 30 to 10 based on early stopping criteria observed during preliminary experimentation, where we found that the CRF-enhanced model converged more quickly than its counterparts. For the other models, particularly the custom transformer-based architectures, a higher epoch count (30 epochs) was maintained to ensure sufficient learning due to their relatively slower convergence rates.

All models share consistent settings for weight decay (0.01), batch size (16), and optimizer choice (AdamW with $\beta = (0.9, 0.95)$). This uniformity allows a more controlled comparison, ensuring that the observed performance differences among models primarily reflect their architectural variations rather than discrepancies in fundamental optimization strategies.

Model Item	BERT_BASE	BERT_CRF	MyBERT_BASE	MyBERT_CRF
learning rate	1e-3	2.5e-4	2.5e-4	3e-5
training epochs	30	10	30	30
weight decay			0.01	
batch size			16	
Optimizer			AdamW: $\beta(0.9, 0.95)$	

Table 2: Parameter settings for different models

3.3 Evaluation Metrics

We use **Recall Rate**, **Precision Rate**, and **F1 Score** to evaluate the ability of these models.¹ These metrics are defined as follows:

Precision measures the fraction of correctly predicted positive observations out of all observations predicted as positive:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (1)$$

Recall quantifies the fraction of correctly identified positive observations out of all actual positive observations:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2)$$

The **F1 Score** is the harmonic mean of precision and recall, providing a balanced measure that considers both metrics simultaneously:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

Each of these metrics emphasizes a different aspect of model performance. **Precision** prioritizes the accuracy of positive predictions, ensuring fewer incorrect entity labels. **Recall** focuses on the model’s sensitivity, aiming to correctly identify as many actual entities as possible. The **F1 Score** strikes a balance between these two metrics. These metrics allow us to comprehensively assess the effectiveness of our models in identifying and classifying named entities.

3.4 Results

The experimental results obtained from evaluating the four model architectures on the CoNLL-2003 test dataset are summarized in Table 3. We report Recall, Precision, and the overall F1 Score to comprehensively analyze each model’s performance.

¹TP denotes true positives, i.e., labels correctly predicted by the model; FP denotes false positives, i.e., labels incorrectly predicted as belonging to the current category; FN denotes false negatives, i.e., labels in the current category that were incorrectly predicted as belonging to other categories.

According to these results, we observe that the pretrained RoBERTa-based models (BERT_BASE and BERT_CRF) significantly outperform the custom transformer architectures (MyBERT_BASE and MyBERT_CRF) across all evaluation metrics. Specifically, the BERT_BASE model achieves the highest recall (83.69%) and the best overall F1 Score (0.8247), indicating its strong capability to capture relevant entities effectively. Meanwhile, the BERT_CRF model achieves higher precision (84.15%), highlighting the benefit of incorporating CRF layers in improving prediction accuracy at the expense of slightly lower recall. (This pattern can also be observed in MyBERT_BASE and MyBERT_CRF).

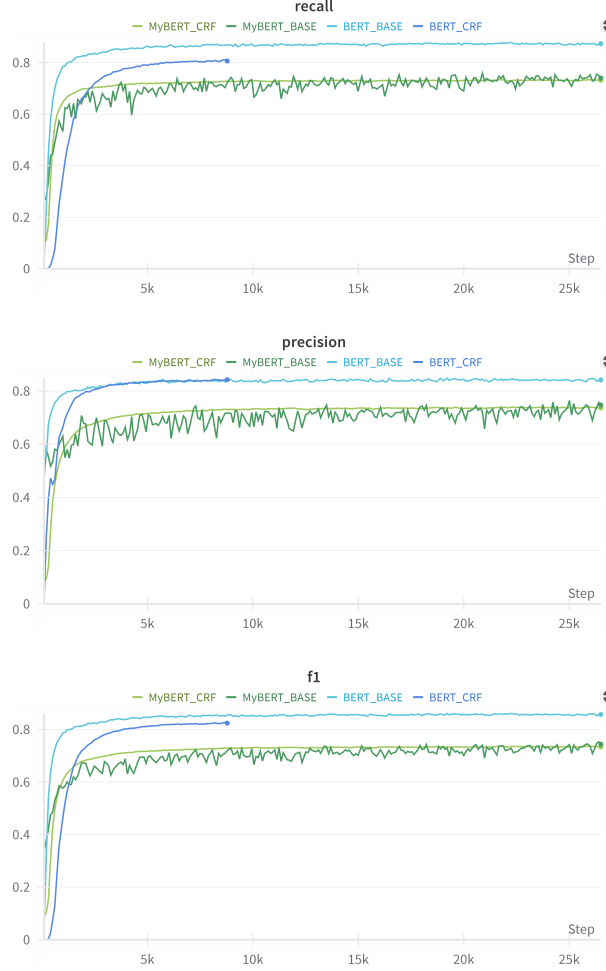


Figure 3: A comparative evaluation curve during training. Top: **Recall** curve of four models. Mid: **Precision** curve of four models. Bottom: **F1 Score** curve of four models.

Comparatively, the custom-designed transformers exhibit considerably lower overall performance, which can be attributed to the lack of extensive pretraining and fewer learned contextual representations. While the addition of a CRF layer in MyBERT_CRF slightly increases precision (64.16%), it simultaneously reduces recall, resulting in a marginally lower overall F1 Score (0.6382) compared to MyBERT_BASE (0.6460). A probable reason is that the CRF layer explicitly models label dependencies, making it more cautious in assigning entity labels, thereby reducing false positives. However, this conservative prediction strategy can sometimes cause the model to overlook true entities, slightly reducing recall.

We also studied what kind of errors are common in this NER task and how they are distributed across entity types (see 6). Among all of the prediction results of the four models, the most common error is that the model mispredicts **B-ORG** as **B-LOC**. A possible reason for this is that many entities in the **B-ORG** category are names of countries or regions, which inherently possess multiple semantic interpretations. Determining whether these entities represent organizational institutions or geographic

Word	NER tag	Label	Word	NER tag	Label
Takuya	B-PER	B-PER	Takuya	B-PER	B-PER
Takagi	I-PER	I-PER	Takagi	I-PER	I-PER
scored	0	0	scored	0	0
the	0	0	the	0	0
winner	0	0	winner	0	0
in	0	0	in	0	0
the	0	0	the	0	0
88th	0	0	88th	0	0
minute	0	0	minute	0	0
,	0	0	,	0	0
rising	0	0	rising	0	0
to	0	0	to	0	0
head	0	0	head	0	0
a	0	0	a	0	0
Hiroshige	B-PER	B-PER	Hiroshige	B-PER	B-PER
Yanagimoto	I-PER	I-PER	Yanagimoto	I-PER	I-PER
cross	0	0	cross	0	0
towards	0	0	towards	0	0
the	0	0	the	0	0
Syrian	B-LOC	B-MISC	Syrian	0	B-MISC
goal	0	0	goal	0	0
which	0	0	which	0	0
goalkeeper	0	0	goalkeeper	0	0
Salem	B-PER	B-PER	Salem	B-PER	B-PER
Bitar	I-PER	I-PER	Bitar	I-PER	I-PER
appeared	0	0	appeared	0	0
to	0	0	to	0	0
have	0	0	have	0	0
covered	0	0	covered	0	0
but	0	0	but	0	0
then	0	0	then	0	0
allowed	0	0	allowed	0	0
to	0	0	to	0	0
slip	0	0	slip	0	0
into	0	0	into	0	0
the	0	0	the	0	0
net	0	0	net	0	0
.	0	0	.	0	0

Figure 4: A typical comparison of the different prediction style between BERT_BASE and BERT_CRF. Left: Prediction of BERT_BASE. Right: Prediction of BERT_CRF. When predicting the tag of word "Syrian", BERT_BASE pointed out it's an entity but wrongly predicted it into **B-LOC**, while BERT_CRF conservatively thought it's not an entity.

Model	Recall (%)	Precision (%)	F1
BERT_BASE	83.69	81.28	0.8247
BERT_CRF	79.64	84.15	0.8183
MyBERT_BASE	66.08	63.18	0.6460
MyBERT_CRF	63.49	64.16	0.6382

Table 3: Experiment results on test dataset

locations often heavily relies on contextual cues, making them particularly challenging for models to classify accurately. Apart from this, we observe noticeable performance differences between the pretrained RoBERTa-based models (BERT_BASE and BERT_CRF) and the custom transformer models (MyBERT_BASE and MyBERT_CRF). Pretrained models generally demonstrate significantly lower error ratios across all entity categories, benefiting from richer contextual embeddings obtained through extensive pretraining. In contrast, the custom models, lacking such comprehensive linguistic knowledge, exhibit substantially higher error rates, particularly in categories such as **X-MISC**, **X-LOC**, and **X-ORG**, suggesting limited capability in accurately modeling nuanced semantic relationships. Additionally, integrating a CRF layer influences the types and distribution of errors. Models with CRF (BERT_CRF and MyBERT_CRF) typically show higher precision, as indicated by fewer false-positive errors. For all models, the biggest challenge is to predict the **I-MISC** tags, which suggests that the identification of intermediate or non-boundary parts of miscellaneous entities remains particularly problematic. This difficulty may stem from the inherent ambiguity and diverse semantic characteristics of miscellaneous entities, which often lack the clear-cut syntactic and contextual markers present in more well-defined entity types. Consequently, even with advanced contextualization and structured prediction techniques, accurately capturing the full extent of such entities remains an open challenge for current NER systems.

	O	B-PER	I-PER	B-ORG	I-ORG	B-LOC	I-LOC	B-MISC	I-MISC	Error Rate (%)
O	-	19	6	47	58	28	14	78	78	0.86
B-PER	61	-	24	33	4	18	0	0	0	8.66
I-PER	2	16	-	0	26	0	0	0	1	3.89
B-ORG	86	19	2	-	12	155	0	47	0	19.33
I-ORG	40	1	7	9	-	8	32	1	9	12.81
B-LOC	67	10	1	94	5	-	1	27	1	12.35
I-LOC	6	0	3	0	37	4	-	0	5	21.4
B-MISC	72	2	0	44	3	39	0	-	8	23.93
I-MISC	28	0	0	0	19	1	6	4	-	27.78

Figure 5: An example of prediction error analysis(Model: BERT_BASE). The rows correspond to the actual labels and the columns correspond to the prediction results.

4 Conclusion

In this project, we implemented an experiment on NER by comparing transformer-based architectures with and without an integrated CRF layer. Our experimental results on the CoNLL-2003 dataset indicate that pretrained models (BERT_BASE and BERT_CRF) significantly outperform custom transformer architectures (MyBERT_BASE and MyBERT_CRF), underscoring the importance of extensive pretraining for capturing rich contextual information.

The introduction of a CRF layer proved beneficial in improving precision by effectively modeling sequential dependencies between entity labels. However, this improvement in precision sometimes came at the expense of recall—a trade-off that is critical in many downstream applications where identifying as many relevant entities as possible is more important than achieving extremely high precision. In practical scenarios such as information retrieval or question answering, missing an entity could lead to incomplete or misleading results. Thus, this result can explain why transformer + CRF is not popular nowadays in NER studies.

References

- [1] Laura Chiticariu, Rajasekar Krishnamurthy, Yunyao Li, Frederick Reiss, and Shivakumar Vaithyanathan. Domain adaptation of rule-based annotators for named-entity recognition tasks. In *Proceedings of the 2010 conference on empirical methods in natural language processing*, pages 1002–1012, 2010.
- [2] Tome Eftimov, Barbara Koroušić Seljak, and Peter Korošec. A rule-based named-entity recognition method for knowledge extraction of evidence-based dietary recommendations. *PloS one*, 12(6):e0179488, 2017.
- [3] Rrubaa Panchendrarajan and Aravindh Amaresan. Bidirectional lstm-crf for named entity recognition. 32nd Pacific Asia Conference on Language, Information and Computation, 2018.
- [4] Ling Luo, Zhihao Yang, Pei Yang, Yin Zhang, Lei Wang, Hongfei Lin, and Jian Wang. An attention-based bilstm-crf approach to document-level chemical named entity recognition. *Bioinformatics*, 34(8):1381–1388, 2018.
- [5] Kaixin Liu, Qingcheng Hu, Jianwei Liu, and Chunxiao Xing. Named entity recognition in chinese electronic medical records based on crf. In *2017 14th Web Information Systems and Applications Conference (WISA)*, pages 105–110. IEEE, 2017.
- [6] Mikhail V Koroteev. Bert: a review of applications in natural language processing and understanding. *arXiv preprint arXiv:2103.11943*, 2021.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [8] Yuxuan Chen, Jonas Mikkelsen, Arne Binder, Christoph Alt, and Leonhard Hennig. A comparative study of pre-trained encoders for low-resource named entity recognition. *arXiv preprint arXiv:2204.04980*, 2022.

- [9] Rongen Yan, Xue Jiang, and Depeng Dang. Named entity recognition by using xlnet-bilstm-crf. *Neural Processing Letters*, 53(5):3339–3356, 2021.

A Appendix

This paper was written manually and polished by the LLM.

```

Label with max error ratio: I-MISC, max error ratio: 0.2778
Label: 0, Total: 38323, Errors: 328, Error Ratio: 0.0086, Most Common Error: B-MISC
Error Details:{'B-MISC': 78, 'I-MISC': 78, 'I-ORG': 58, 'B-ORG': 47, 'B-PER': 19, 'B-LOC': 28, 'I-LOC': 14, 'I-PER': 6}
Label: B-PER, Total: 1617, Errors: 140, Error Ratio: 0.0866, Most Common Error: 0
Error Details:{'O': 61, 'B-ORG': 33, 'B-LOC': 18, 'I-PER': 24, 'I-ORG': 4}
Label: I-PER, Total: 1156, Errors: 45, Error Ratio: 0.0389, Most Common Error: I-ORG
Error Details:{'B-PER': 16, 'I-ORG': 26, 'O': 2, 'I-MISC': 1}
Label: B-ORG, Total: 1661, Errors: 321, Error Ratio: 0.1933, Most Common Error: B-LOC
Error Details:{'O': 86, 'B-LOC': 155, 'I-ORG': 12, 'B-MISC': 47, 'B-PER': 19, 'I-PER': 2}
Label: I-ORG, Total: 835, Errors: 107, Error Ratio: 0.1281, Most Common Error: 0
Error Details:{'B-ORG': 9, 'O': 40, 'I-PER': 7, 'I-LOC': 32, 'I-MISC': 9, 'B-LOC': 8, 'B-PER': 1, 'B-MISC': 1}
Label: B-LOC, Total: 1668, Errors: 206, Error Ratio: 0.1235, Most Common Error: B-ORG
Error Details:{'B-ORG': 94, 'O': 67, 'B-PER': 10, 'B-MISC': 27, 'I-ORG': 5, 'I-LOC': 1, 'I-MISC': 1, 'I-PER': 1}
Label: I-LOC, Total: 257, Errors: 55, Error Ratio: 0.2140, Most Common Error: I-ORG
Error Details:{'I-ORG': 37, 'I-PER': 3, 'B-LOC': 4, 'I-MISC': 5, 'O': 6}
Label: B-MISC, Total: 702, Errors: 168, Error Ratio: 0.2393, Most Common Error: 0
Error Details:{'O': 72, 'I-MISC': 8, 'B-LOC': 39, 'B-ORG': 44, 'I-ORG': 3, 'B-PER': 2}
Label: I-MISC, Total: 216, Errors: 60, Error Ratio: 0.2778, Most Common Error: 0
Error Details:{'O': 28, 'B-LOC': 1, 'I-ORG': 19, 'B-MISC': 4, 'I-LOC': 6, 'I-PER': 2}

Label with max error ratio: I-MISC, max error ratio: 0.3611
Label: 0, Total: 38323, Errors: 246, Error Ratio: 0.0064, Most Common Error: B-ORG
Error Details:{'B-PER': 24, 'B-ORG': 67, 'B-LOC': 28, 'B-MISC': 36, 'I-ORG': 37, 'I-PER': 8, 'I-MISC': 36, 'I-LOC': 10}
Label: B-PER, Total: 1617, Errors: 191, Error Ratio: 0.1181, Most Common Error: 0
Error Details:{'O': 93, 'B-ORG': 52, 'I-PER': 28, 'B-LOC': 14, 'I-ORG': 2, 'I-MISC': 1, 'B-MISC': 1}
Label: I-PER, Total: 1156, Errors: 51, Error Ratio: 0.0441, Most Common Error: I-ORG
Error Details:{'B-PER': 15, 'I-ORG': 24, 'O': 10, 'I-LOC': 1, 'I-MISC': 1}
Label: B-ORG, Total: 1661, Errors: 365, Error Ratio: 0.2197, Most Common Error: 0
Error Details:{'O': 170, 'B-LOC': 139, 'I-ORG': 12, 'B-MISC': 32, 'B-PER': 11, 'I-PER': 1}
Label: I-ORG, Total: 835, Errors: 140, Error Ratio: 0.1677, Most Common Error: 0
Error Details:{'B-LOC': 10, 'I-LOC': 33, 'O': 65, 'I-MISC': 16, 'B-ORG': 6, 'I-PER': 7, 'B-MISC': 2, 'B-PER': 1}
Label: B-LOC, Total: 1668, Errors: 280, Error Ratio: 0.1679, Most Common Error: B-ORG
Error Details:{'O': 124, 'B-ORG': 125, 'B-MISC': 21, 'B-PER': 7, 'I-PER': 1, 'I-LOC': 1, 'I-ORG': 1}
Label: I-LOC, Total: 257, Errors: 55, Error Ratio: 0.2140, Most Common Error: I-ORG
Error Details:{'I-ORG': 30, 'I-MISC': 2, 'O': 17, 'I-PER': 3, 'B-MISC': 2, 'B-LOC': 1}
Label: B-MISC, Total: 702, Errors: 246, Error Ratio: 0.3504, Most Common Error: 0
Error Details:{'O': 157, 'B-LOC': 33, 'B-ORG': 45, 'I-MISC': 5, 'I-ORG': 2, 'B-PER': 4}
Label: I-MISC, Total: 216, Errors: 78, Error Ratio: 0.3611, Most Common Error: 0
Error Details:{'O': 53, 'I-LOC': 5, 'I-ORG': 15, 'B-LOC': 2, 'B-MISC': 3}

Label with max error ratio: I-MISC, max error ratio: 0.4352
Label: 0, Total: 38323, Errors: 638, Error Ratio: 0.0166, Most Common Error: B-LOC
Error Details:{'I-PER': 91, 'B-LOC': 158, 'I-MISC': 65, 'B-ORG': 71, 'B-MISC': 68, 'B-PER': 79, 'I-LOC': 28, 'I-ORG': 78}
Label: B-PER, Total: 1617, Errors: 556, Error Ratio: 0.3438, Most Common Error: 0
Error Details:{'O': 247, 'I-PER': 85, 'B-MISC': 24, 'B-LOC': 102, 'I-ORG': 11, 'B-ORG': 80, 'I-MISC': 1, 'I-LOC': 6}
Label: I-PER, Total: 1156, Errors: 250, Error Ratio: 0.2163, Most Common Error: 0
Error Details:{'B-MISC': 6, 'O': 117, 'I-ORG': 46, 'B-LOC': 15, 'B-PER': 36, 'I-LOC': 13, 'B-ORG': 16, 'I-MISC': 1}
Label: B-ORG, Total: 1661, Errors: 674, Error Ratio: 0.4058, Most Common Error: 0
Error Details:{'B-LOC': 195, 'O': 275, 'B-MISC': 52, 'I-MISC': 6, 'I-PER': 32, 'I-ORG': 34, 'I-LOC': 3, 'B-PER': 77}
Label: I-ORG, Total: 835, Errors: 334, Error Ratio: 0.4000, Most Common Error: 0
Error Details:{'O': 148, 'B-LOC': 40, 'I-PER': 57, 'I-LOC': 41, 'B-PER': 12, 'B-ORG': 9, 'B-MISC': 15, 'I-MISC': 12}
Label: B-LOC, Total: 1668, Errors: 264, Error Ratio: 0.1583, Most Common Error: 0
Error Details:{'O': 110, 'B-ORG': 76, 'B-PER': 24, 'I-PER': 13, 'B-MISC': 23, 'I-ORG': 12, 'I-LOC': 5, 'I-MISC': 1}
Label: I-LOC, Total: 257, Errors: 99, Error Ratio: 0.3852, Most Common Error: 0
Error Details:{'I-ORG': 33, 'O': 34, 'I-PER': 11, 'B-LOC': 17, 'B-PER': 1, 'B-MISC': 1, 'I-MISC': 2}
Label: B-MISC, Total: 702, Errors: 232, Error Ratio: 0.3305, Most Common Error: 0
Error Details:{'B-LOC': 63, 'B-PER': 20, 'O': 102, 'B-ORG': 29, 'I-ORG': 6, 'I-MISC': 5, 'I-PER': 6, 'I-LOC': 1}
Label: I-MISC, Total: 216, Errors: 94, Error Ratio: 0.4352, Most Common Error: 0
Error Details:{'O': 54, 'I-ORG': 6, 'I-PER': 7, 'B-MISC': 19, 'B-PER': 5, 'B-LOC': 1, 'I-LOC': 2}

Label with max error ratio: I-LOC, max error ratio: 0.4669
Label: 0, Total: 38323, Errors: 470, Error Ratio: 0.0123, Most Common Error: B-ORG
Error Details:{'B-ORG': 114, 'I-PER': 40, 'I-ORG': 56, 'B-PER': 81, 'B-LOC': 75, 'B-MISC': 43, 'I-MISC': 48, 'I-LOC': 13}
Label: B-PER, Total: 1617, Errors: 545, Error Ratio: 0.3370, Most Common Error: 0
Error Details:{'B-ORG': 90, 'O': 300, 'B-LOC': 52, 'B-MISC': 11, 'I-PER': 71, 'I-ORG': 13, 'I-LOC': 2, 'I-MISC': 6}
Label: I-PER, Total: 1156, Errors: 360, Error Ratio: 0.3114, Most Common Error: 0
Error Details:{'I-ORG': 39, 'O': 206, 'B-ORG': 23, 'B-MISC': 11, 'B-PER': 61, 'B-LOC': 6, 'I-LOC': 5, 'I-MISC': 9}
Label: B-ORG, Total: 1661, Errors: 581, Error Ratio: 0.3498, Most Common Error: 0
Error Details:{'B-MISC': 36, 'O': 322, 'B-PER': 79, 'B-LOC': 102, 'I-PER': 14, 'I-ORG': 22, 'I-MISC': 4, 'I-LOC': 2}
Label: I-ORG, Total: 835, Errors: 351, Error Ratio: 0.4204, Most Common Error: 0
Error Details:{'O': 206, 'I-PER': 36, 'B-PER': 13, 'B-LOC': 21, 'B-ORG': 27, 'I-LOC': 22, 'B-MISC': 15, 'I-MISC': 11}
Label: B-LOC, Total: 1668, Errors: 388, Error Ratio: 0.2326, Most Common Error: 0
Error Details:{'B-ORG': 130, 'O': 163, 'I-PER': 9, 'I-ORG': 16, 'B-PER': 35, 'I-MISC': 2, 'B-MISC': 25, 'I-LOC': 8}
Label: I-LOC, Total: 257, Errors: 120, Error Ratio: 0.4669, Most Common Error: 0
Error Details:{'O': 54, 'I-PER': 10, 'B-LOC': 14, 'I-ORG': 34, 'I-MISC': 2, 'B-ORG': 4, 'B-PER': 2}
Label: B-MISC, Total: 702, Errors: 242, Error Ratio: 0.3447, Most Common Error: 0
Error Details:{'B-PER': 20, 'O': 126, 'I-ORG': 4, 'B-LOC': 33, 'B-ORG': 52, 'I-MISC': 5, 'I-PER': 2}
Label: I-MISC, Total: 216, Errors: 95, Error Ratio: 0.4398, Most Common Error: 0
Error Details:{'O': 59, 'B-PER': 6, 'B-MISC': 19, 'I-ORG': 8, 'I-PER': 2, 'I-LOC': 1}

```

Figure 6: Detailed Error Analysis results of four models. From top to the bottom: BERT_BASE, BERT_CRF, MyBERT_BASE, MyBERT_CRF.